

Sai

Searching

- Most Common operation
- Time - Consuming operation
- finds location of the target in a list of objects
- Sequential search
- Binary search
- Hashed list search
- Can be applied to arrays or linked lists

```
int SeqSearch (int arr[], int length, int x)
{
    int i = 0;
    for (i = 0; i < length; i = i + 1)
        if (arr[i] == x)
            return (i);
}
```

```
    return(-1);  
}
```

```
int SentinelSearch (int arr[], int length, int x)  
{ // assumption arr is of length = length + 1  
    int i = 0;  
    arr[length] = x;  
    while (arr[i] != x)  
        i = i + 1;  
    if (i < length)  
        return(i);  
    return(-1);  
}
```

Probability Search :

- Most probable in the beginning
- Least probable at the end
- In each search, exchange the located element with the element before it.


```

int f = 0, l = length - 1, m = 0;
while (f <= l)
{
    m = (f + l) / 2;
    if (x < arr[m])
        l = m - 1;
    else if (x > arr[m])
        f = m + 1;
    else
        return (m);
}
return (-1);
}

```

Sequential Search : $O(n)$

Binary Search : $O(\log n)$

Hashed List Search

→ Key determines the location

→ Function used to transform key to location → Hashing function
 e.g. Store student details in array in increasing order of reg. no.

struct Student

```

{
  int regno;
  char name[20];
  char class[10];
  int age;
};
  
```

e.g. int arr[33];

→ struct Student
 students[33];
 Memory to store
 33 student structure

Students

0	1	32
RegNo: 234201 Name: Sreekan class: I B.S. Age: 17	Abhishek	RegNo: 234233 Name: T. Pavan class: I B.S. Age: 17

Given a reg no., find student details.

$$h(\text{regno}) = \text{regno} - 234201$$

$O(1)$ → constant

51	52	-	28	.	100
----	----	---	----	---	-----

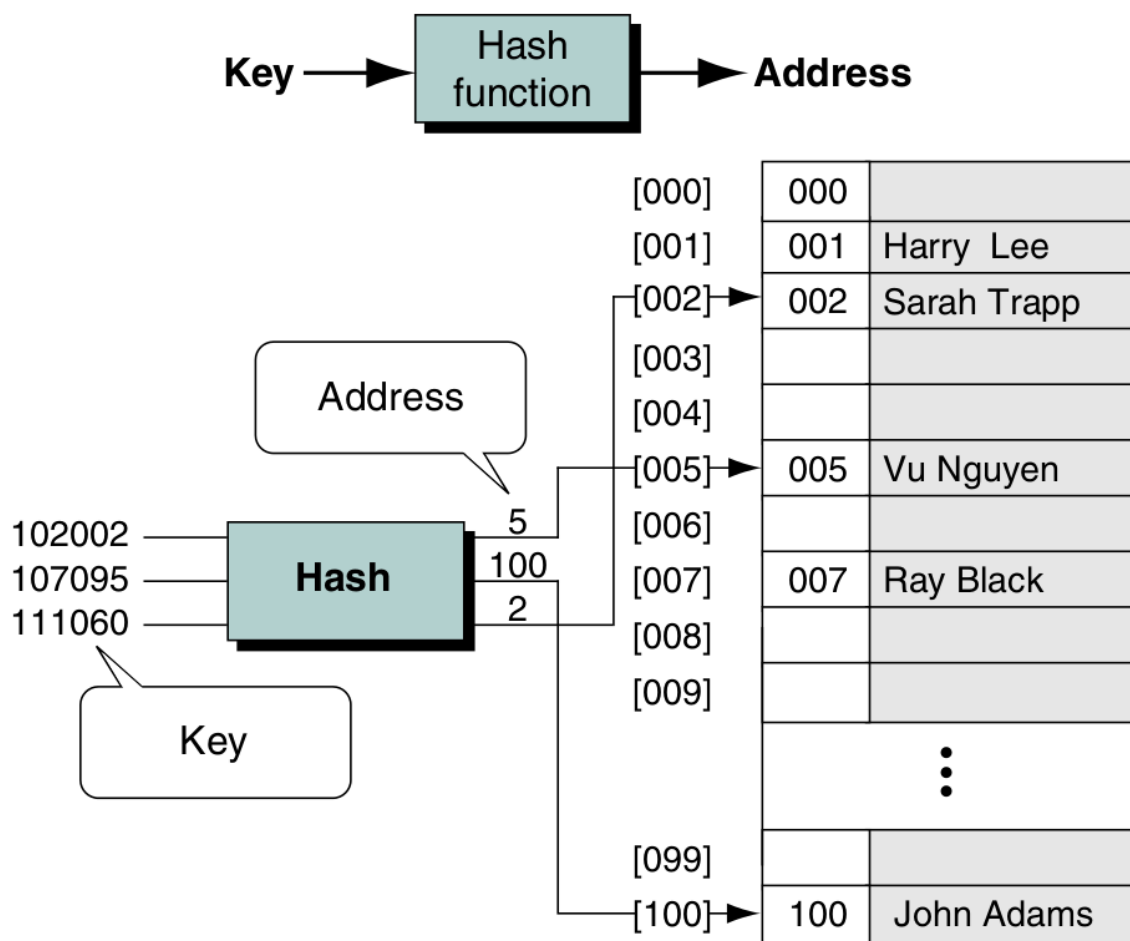
↓

Where is 79?

$$h(x) = x - 51$$

$$h(79) = 79 - 51$$

$$= \underline{\underline{28}}$$



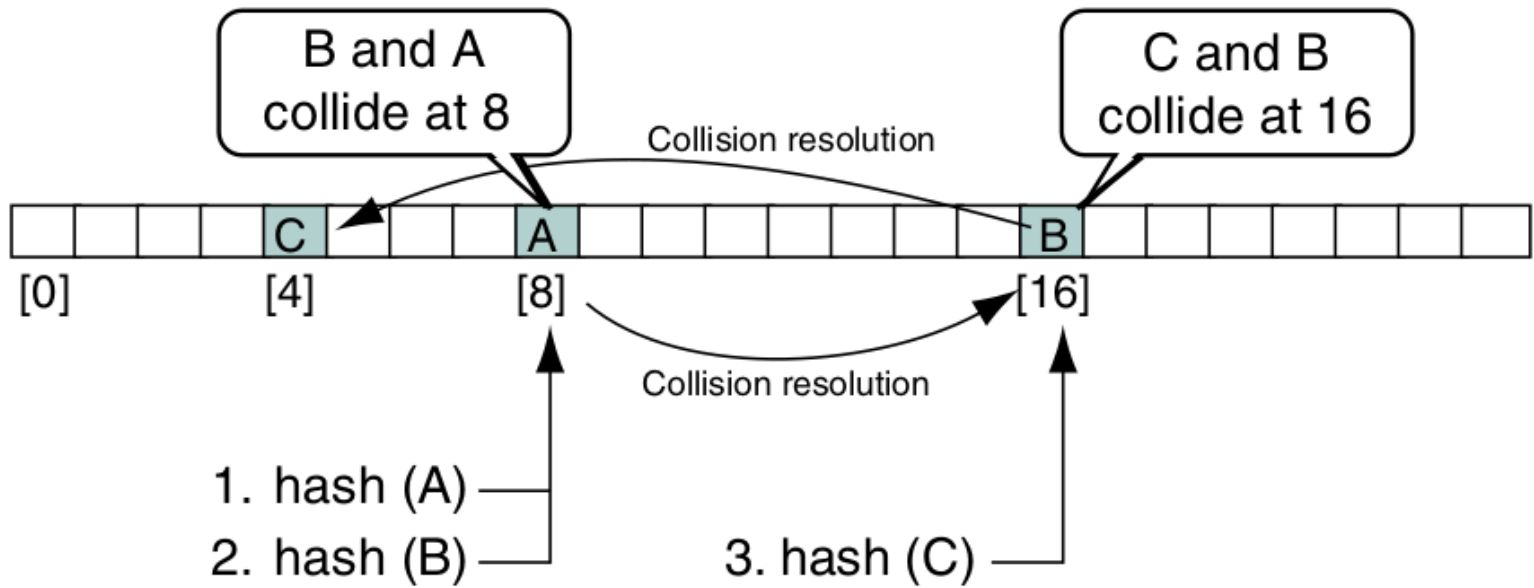
→ Generally, No. of keys $\gg$ Storage area
 ⇒ More than one key can hash to the same location → Synonyms.

→ Collision: When a hashing algorithm produces an address for a key and that address is already occupied.

→ Home Address: Address produced by the hashing algorithm.

→ Prime area: Memory that contains all of the home addresses.

Collision Resolution : When two keys collide at a home address, resolve the collision by placing one of the keys and its data in another location .



A & B are synonyms : T / F ?

B & C are synonyms : T / F ?

→ Obviously, same hashing algorithm used to locate an element must be used when an element is inserted.

→ ∴ Hashed search involves :

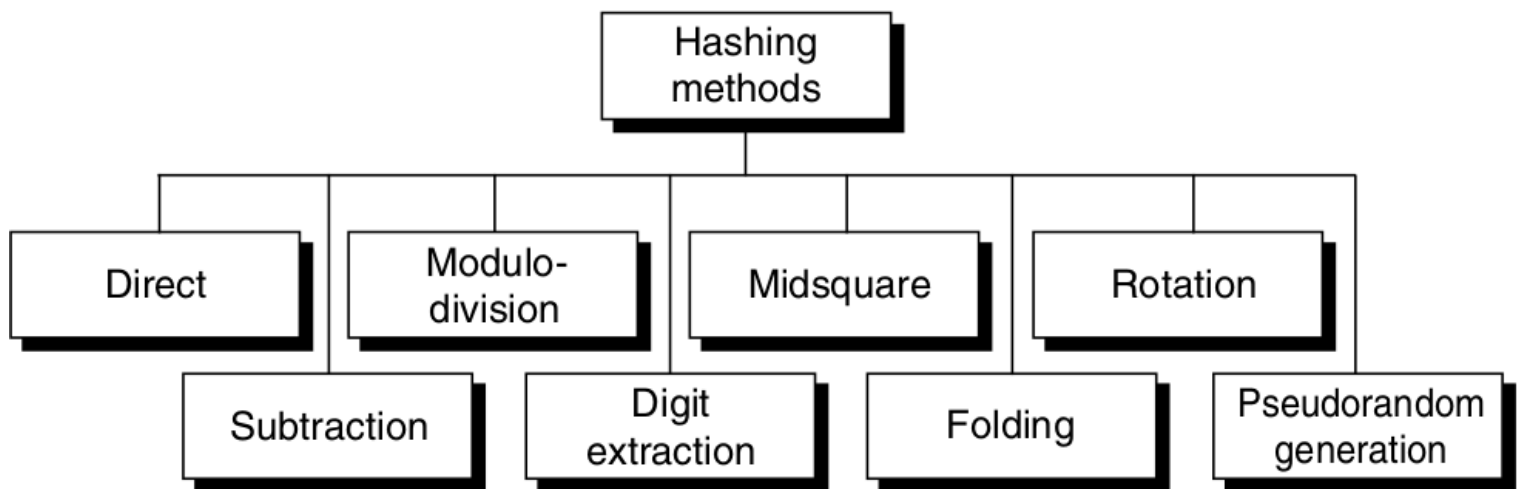
- home address $\leftarrow$ hash(key)
- if home address contains key
return home address [location]
- else
Use Collision Resolution to determine next location

if (found)
return location

{ Continue this process until we
find the element or determine
that it is not in the list }

Probe : Each calculation of an address and
test for success.

Hashing Methods



Direct hashing : key is the address

e.g. day Sales $\text{Sales}[\text{day}] += \text{Amount};$

$1 \leq \text{day} \leq 31$

Subtraction Method: If keys are Consecutive but do not start from 1.

e.g- 234201 to 234233.

Both Direct & Subtraction:

→ Simple

→ No collisions

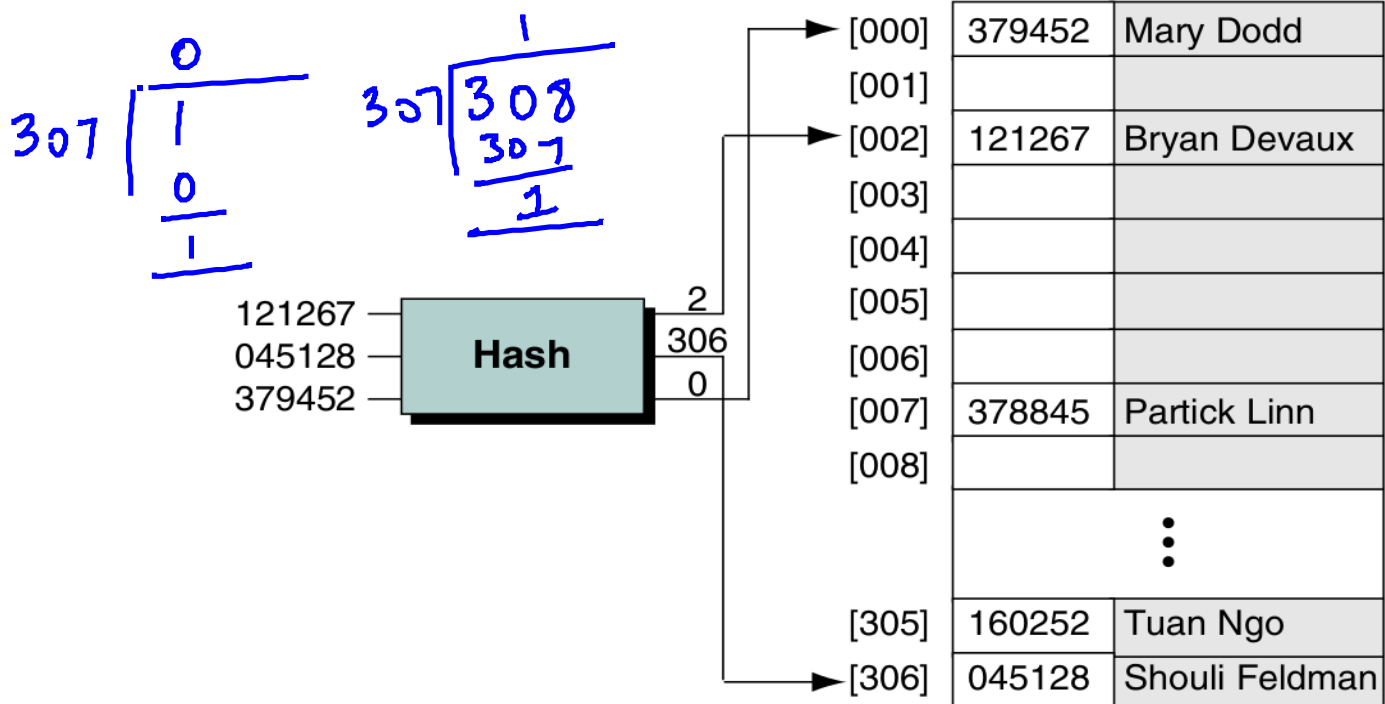
→ One-to-one hashing

→ But Can be used only for keys that are Consecutive.

Modulo - Division Method:

address = key modulo list size

→ If list size is prime, it produces lesser collision.



Digit - Extraction Method: Selected digits are extracted from the key and used as the address.

379452 $\Rightarrow$ 394

121267 $\Rightarrow$ 112

378845 $\Rightarrow$ 388

160252 $\Rightarrow$ 102

045128 $\Rightarrow$ 051

→ How many storage locations?

Midsquare Method: Key is squared and address is selected from the middle of the squared number.

→ If too large, select a portion of the key and not entire key.

379452: $379^2 = 143641 \Rightarrow 364$

121267: $121^2 = 014641 \Rightarrow 464$

378845: $378^2 = 142884 \Rightarrow 288$

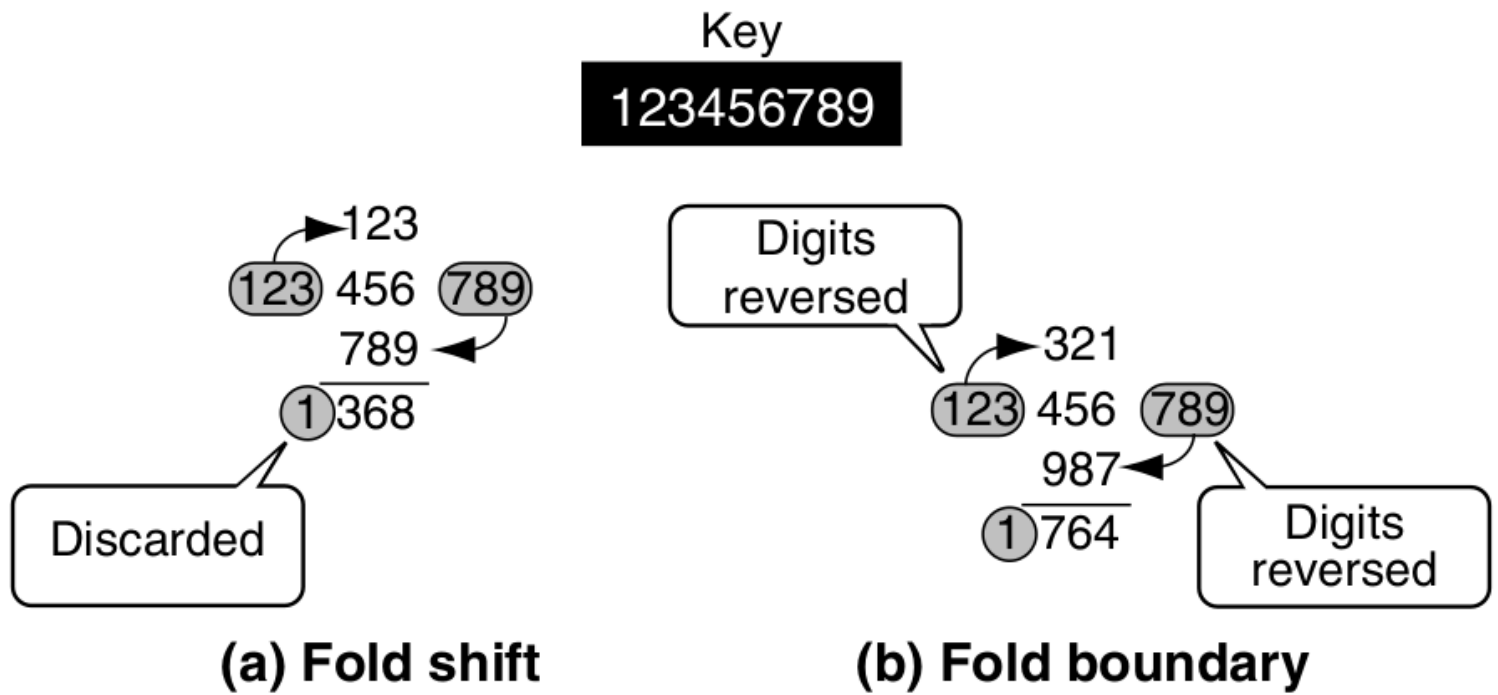
160252: $160^2 = 025600 \Rightarrow 560$

045128: $045^2 = 002025 \Rightarrow 202$

Fold Shift: Key value is divided into parts whose size matches size of the address.

Then, left and right parts are shifted and added with the middle part.

Fold Boundary: Left and right numbers are folded on a fixed boundary between them and the center number.



Rotation Method: Usually used in combination with other methods.

→ Most useful when keys are assigned serially

600101 → Fold shift with 2 digits → 60 62
 600102 → 01 63
 600103 → 01 64
 600104 → 62 65
 600105 → 62 66
 (Sequential)

600101
600102
600103
600104
600105

Rotated key
→

1 60010
2 60010 → 16
3 60010 00
4 60010 10
5 60010 26

→ 26
36
46
56
66
(Spreads the data)

Pseudorandom Method: Key is used as the seed in a pseudorandom number generator, then scaled into the possible address range.

$$\text{eg. } y = (a x + b) \% c$$

↓
key

→ list size

a, b - prime

```
y = ((17 * 121267) + 7) modulo 307
y = (2061539 + 7) modulo 307
y = 2061546 modulo 307
y = 41
```

All hash functions except direct and subtraction are many-to-one functions.
→ One hashing function implementation
↓

Algorithm hash (key, size, maxAddr, addr)

This algorithm converts an alphanumeric key of size characters into an integral address.

Pre key is a key to be hashed

 size is the number of characters in the key

 maxAddr is maximum possible address for the list

Post addr contains the hashed address

1 set looper to 0

2 set addr to 0

 Hash key

3 for each character in key

 1 if (character not space)

 1 add character to address

 2 rotate addr 12 bits right

 2 end if

4 end loop

 Test for negative address

5 if (addr < 0)

 1 addr = absolute(addr)

6 end if

7 addr = addr modulo maxAddr

end hash

```

int hash(char key[], int len, int size)
{
    int add = 0, i = 0;
    for(i = 0; i < len; i++)
    {
        if (key[i] != '\0')
        {
            add += key[i];
            add = add >> 12;
        }
    }
    if (add < 0)
        add = add * -1;
    return (add % size);
}

```

32 % 17 → 15

Collision Resolution

- Needed for all the many-to-one hashing functions
- Many methods, each of them independent of the hashing algorithm

→ Three concepts needed

→ Load factor

→ Clustering

 ↙ ↘
 Primary Secondary

→ Collision limit

Load factor: Due to possibility of collisions, there must be some empty elements in the list at all times.

→ Full list: All elements except one contain data

Rule of thumb: A hashed list should not be allowed to become more than 75% full.

$$\text{L F} = \frac{\text{No. of elements in the list}}{\text{No. of storage units in the list}} \times 100$$

(Percentage)

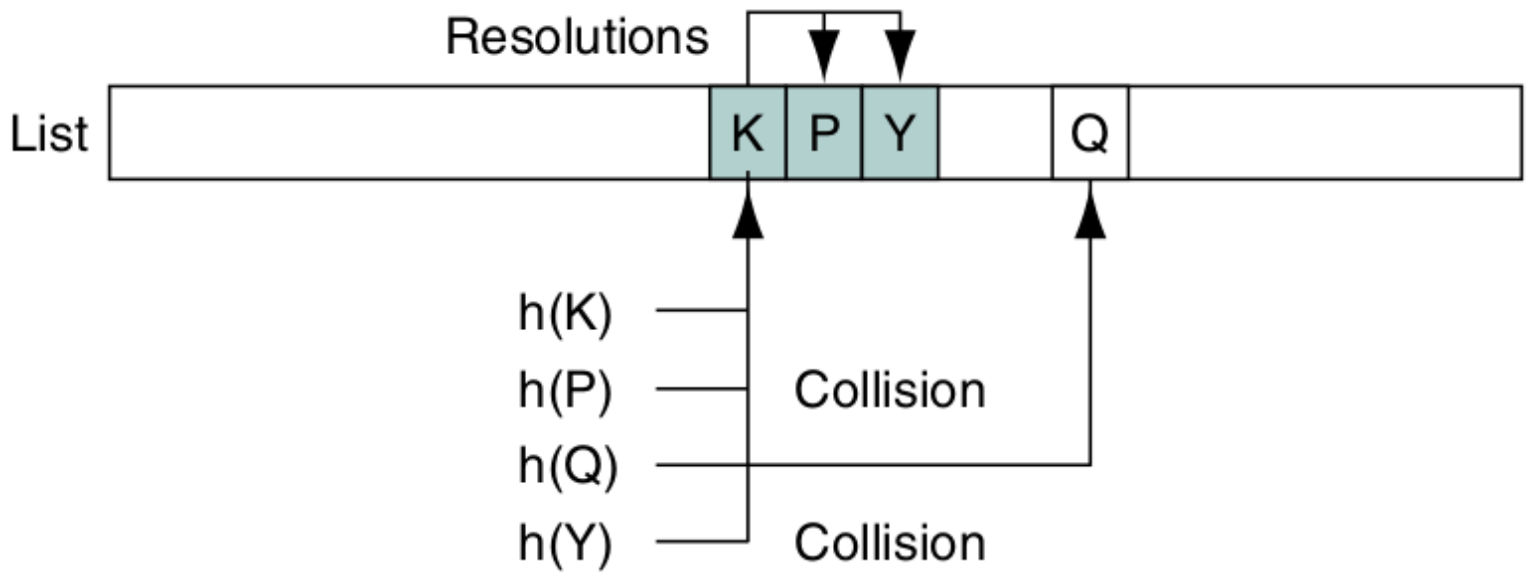
$$\alpha = \frac{k}{n} \times 100$$

Clustering: As collisions occur and are resolved, some hashing algorithms tend to cause data to group within the list.

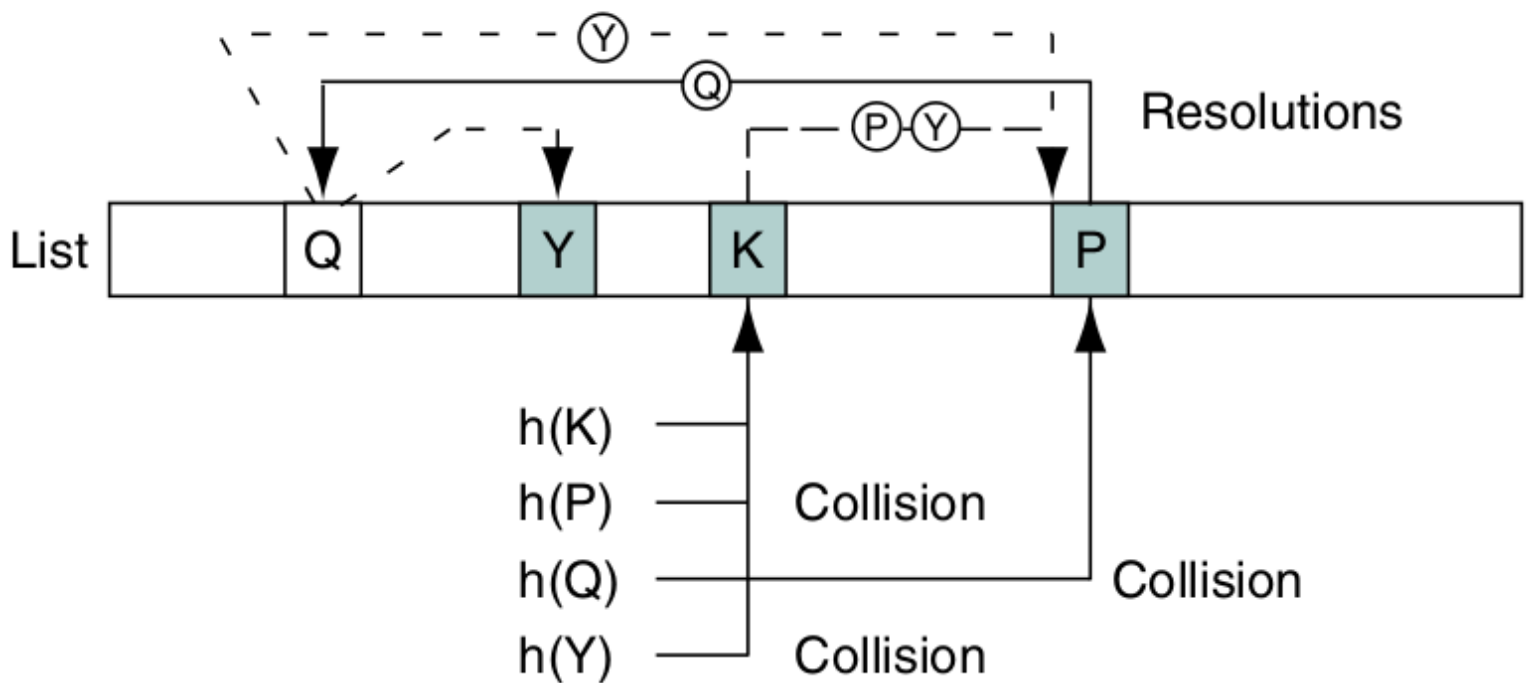
→ Higher the degree of clustering, larger the number of probes to locate an element.

Primary clustering: Data cluster around home address.

Secondary clustering: Data becomes grouped along a collision path.



(a) Primary clustering

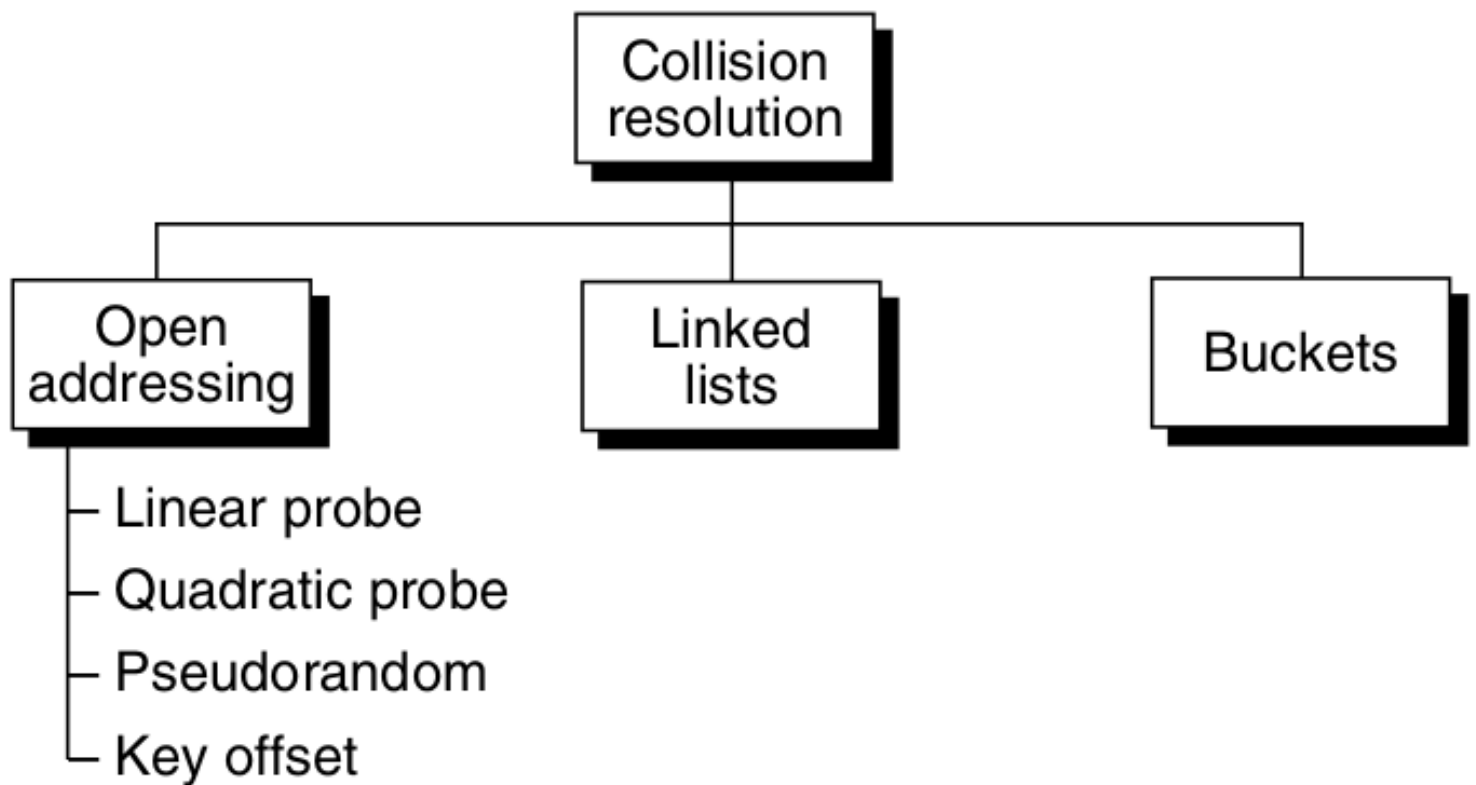


(b) Secondary clustering

⇒ Design hashing algorithms to minimize clustering, both primary and secondary.

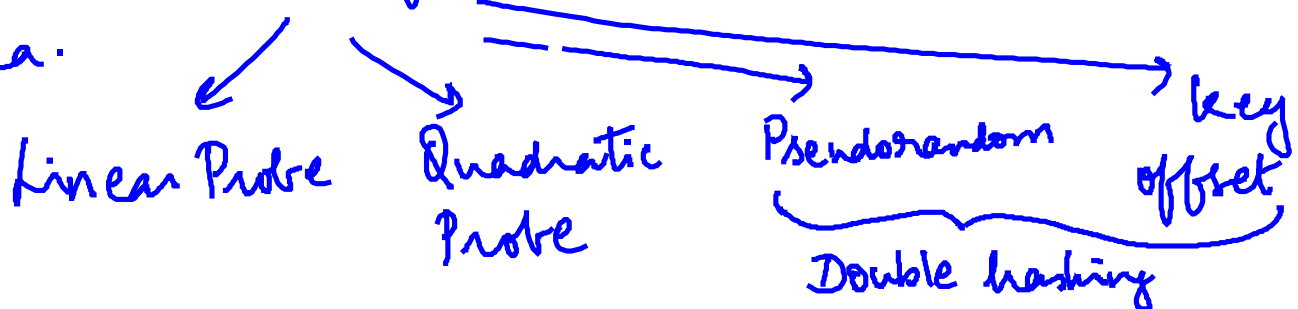
Collision Limit : No. of elements examined in search for a place to store data must be limited.
→ Cannot look at all places to find a place - Very inefficient.

→ But if limit is reached, action is based on the application. e.g. abort

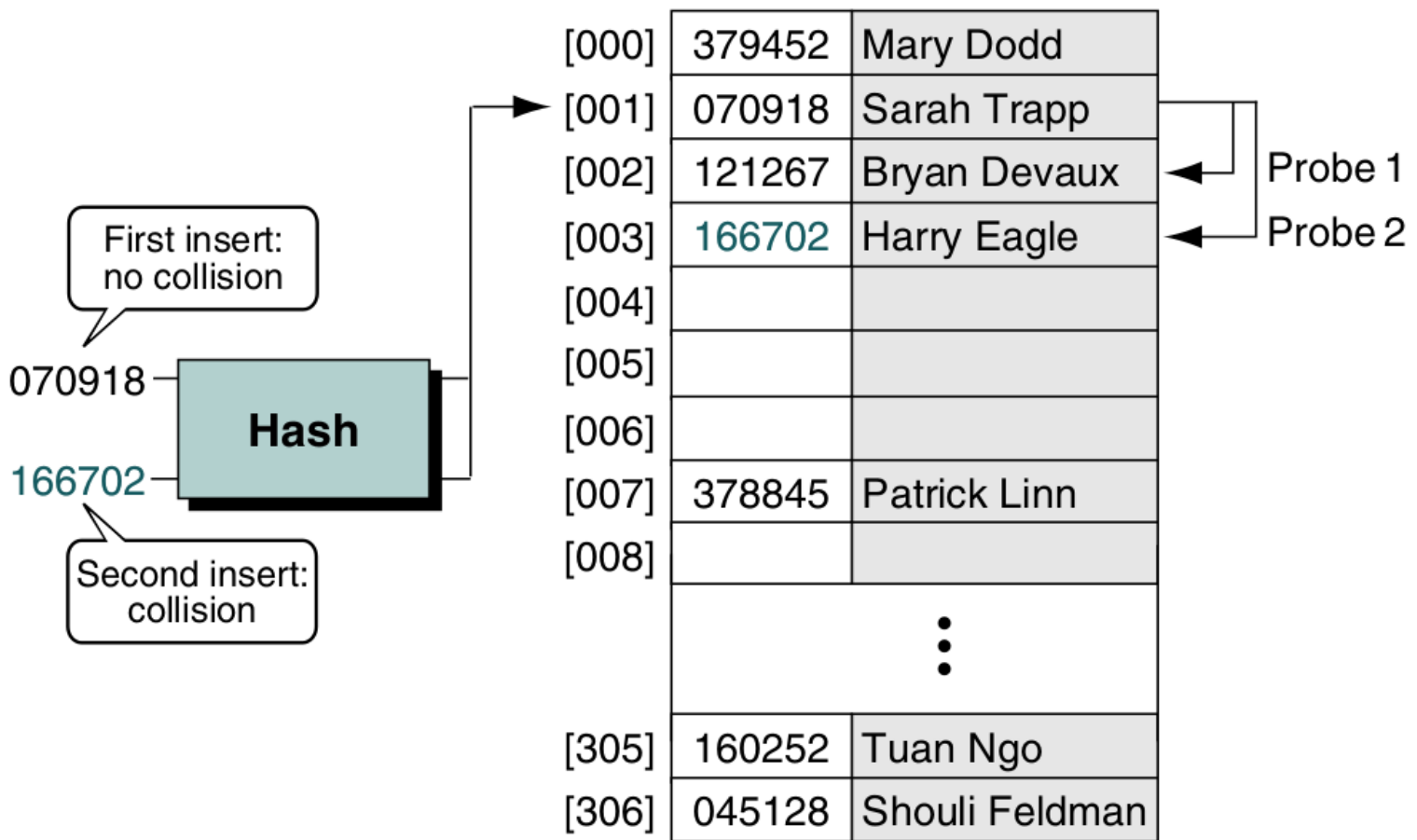


Collision Resolution techniques use two different storage areas → Prime area | overflow area (home addresses)

Open addressing : Resolves collisions in the prime area.



Linear Probe : If there is collision ,
add 1 to current address.



Alternative : add 1, subtract 2, add 3, ...

→ But all operations to ensure location is within the list $\Rightarrow$ last + 1 $\rightarrow$ first
first - 1 $\rightarrow$ last

Advantages:

- Simple to implement
- Data remains near home address

Disadvantages:

- Can produce primary clustering
- Search algorithm becomes complex, especially after deletion

Quadratic probe: Primary clustering can be avoided by adding a value other than 1.

↳ Increment = (Collision Probe Number)²

1st probe → Collision → add 1²
 2nd " " " " add 2² etc.

(Use modulo to be within the list)

For a list size of 100

Probe number	Collision location	Probe ² and increment	New address
1	1	1 ² = 1	1 + 1 ⇒ 02
2	2	2 ² = 4	2 + 4 ⇒ 06
3	6	3 ² = 9	6 + 9 ⇒ 15
4	15	4 ² = 16	15 + 16 ⇒ 31
5	31	5 ² = 25	31 + 25 ⇒ 56
6	56	6 ² = 36	56 + 36 ⇒ 92
7	92	7 ² = 49	92 + 49 ⇒ 41
8	41	8 ² = 64	41 + 64 ⇒ 05
9	5	9 ² = 81	5 + 81 ⇒ 86
10	86	10 ² = 100	86 + 100 ⇒ 86

Advantage: Avoids primary clustering.

Potential Disadvantage: Multiplication every time
 → can be eliminated by incrementing by 2.

$$1 + (1+2) = 4 \quad 4 + (3+2) = 9$$

$$9 + (5+2) = 16 \quad 16 + (7+2) = 25 \text{ etc.}$$

Disadvantage: Not possible to generate a new address for each element in the list.

e.g. For list size 100, only 59 of the probes can generate unique addresses.

$$1 + 1 = 2$$

$$86 + 121 = 207$$

(= 7)

$$2 + 4 = 6$$

$$7 + 144 = 151$$

(= 51)

$$6 + 9 = 15$$

$$15 + 16 = 31$$

$$51 + 169 = 220$$

(= 20)

$$31 + 25 = 56$$

$$20 + 196 = 216$$

(= 16)

$$56 + 36 = 92$$

$$16 + 225 = 241$$

(= 41)

$$92 + 49 = 141$$

(= 41)

$$41 + 64 = 105$$

(= 5)

$$5 + 81 = 86$$

$$86 + 100 = 186$$

(= 86)

Pseudorandom Collision Resolution

→ Like pseudorandom hashing, use pseudorandom number generator to resolve collisions.

→ Instead of the key, use the collision address for the pseudorandom number calculation

e.g. if hashed value = 1, and if 1 is occupied,

$$y = (ax + b) \bmod \text{list size} = (3x + 5) \cdot 307$$

$= (3 \times 1 + 5) \cdot 307 = 8$, hence try next location as 8.

Advantage: Avoids Primary clustering

Disadvantage: Creates secondary clustering
(all keys follow one resolution path)

Key offset Resolution

- Produces different collision paths for different keys
- calculates new address as a function of old address and the key.

e.g. $\text{offset} = \lfloor \text{key} / \text{list size} \rfloor$

$$\text{address} = (\text{offset} + \text{old address}) \bmod \text{list size}$$

Key	Home address	Key offset	Probe 1	Probe 2
166702	1	543	237	166
572556	1	1865	024	047
067234	1	219	220	132

⇒ Each key resolves its collision at a different address for both the first and second probes.

Linked List Collision Resolution

- Major disadvantage of open addressing:
- Each collision resolution increases the probability of future collisions.
 - ⇒ Maintain a linked list of items at each location.
 - ⇒ Uses prime area and overflow area.

[000]	379452	Mary Dodd	X
[001]	070918	Sarah Trapp	→ 166702 Harry Eagle ↓
[002]	121267	Bryan Devaux	X
[003]			X
[004]			X
[005]			X
[006]			X
[007]	378845	Patrick Linn	X
[008]			X
⋮			
[305]	160252	Tuan Ngo	X
[306]	045128	Shouli Feldman	X

166702	Harry Eagle	↓
572556	Chris Walljasper	X

Bucket Hashing Collision Resolution

- Each unit of storage to maintain multiple data units.
- Hence collisions are postponed till a bucket is full.

[000]	Bucket 0	379452	Mary Dodd
[001]	Bucket 1	070918	Sarah Trapp
		166702	Harry Eagle
		367173	Ann Giorgis
[002]	Bucket 2	121267	Bryan Devaux
		572556	Chris Walljasper
		⋮	
[307]	Bucket 307	045128	Shouli Feldman

Linear probe placed here

Linear probe
placed here

But needs to use some or the other Collision Resolution technique when a bucket becomes full.